staunton

Chess diagrams, games and tournament tables for Typst

User manual · package version 0.1.0

All examples assume the package is in scope:

```
#import "@preview/staunton:0.1.0": *
```

In every framed example below, the left side is **the code you type** and the right side is **exactly what it renders** — the manual compiles its own examples, so the two can never disagree.

Contents

1 chess-diagram 2
1.1 Board labels 2
2 Highlights and arrows 3
3 board and diagram 4
4 position 4
5 Games (PGN) 5
5.1 Locators 6
5.2 Playing moves onto a position 6
5.3 Notation output 7
5.4 Exporting FEN 8
5.5 Drawing annotations 8
5.6 Errors 8
6 Tournament tables 8
7 Document-wide style 9
7.1 Language 10
7.2 PGN handling 10
8 Outlines and references 10
9 Pieces and fonts 11
10 Coordinates 11
11 Sizing 11

1 chess-diagram

`chess-diagram(source, ..)` is the everyday entry point for **standard western chess**. It returns a `#figure` with `kind: "chess"`, so it is captioned and cross-referenceable. The simplest call takes a FEN string:

```
#chess-diagram(
  "rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/
RNBQKBNR",
  size: 4cm,
)
```

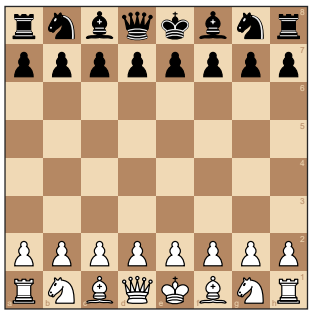


Diagram 1: Position at move 1, White to play

`source` is one of: a **FEN string**; a **position** dict from `position(..)` or `parse-fen(..)`; or a bare **squares** dict (square name → `(kind, color)`).

A diagram can carry two labels: a **game-info** line above the board (drawn automatically as `"<White> – <Black> (<Year>)"` when both players are known), and the figure **caption** below it.

```
#chess-diagram(
  "r1bqkbnr/pppp1ppp/2n5/4p3/4P3/5N2/
PPPP1PPP/RNBQKB1R",
  white: "Morphy", black: "NN", year:
1858,
  caption: [After 2...Nc6],
  size: 4.2cm,
)
```

Morphy – NN (1858)

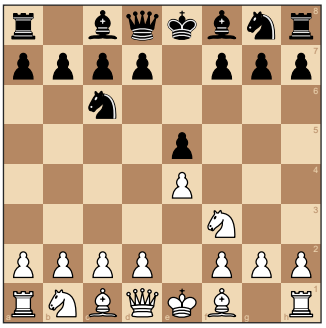


Diagram 2: After 2...Nc6

Use `flip: true` to show the board from Black's side, and `piece-set:` to pick a bundled set (`"cburnett"` default, `"merida"`, or `"unicode"` for the glyph fallback):

```
#chess-diagram(
  "8/8/8/3k4/3K4/8/8/8",
  flip: true,
  piece-set: "merida",
  size: 3.6cm,
)
```

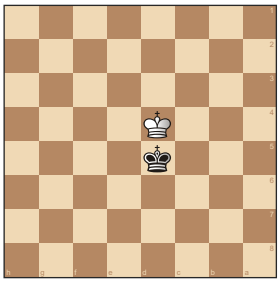


Diagram 3: Position at move 1, White to play

1.1 Board labels

`label-mode` chooses how files and ranks are drawn — `"on-square"` (default, tucked into the corner squares), `"outside"` (a gutter strip), or `"border"` (a themed band, styled by `border-theme`). `labels: false` suppresses them.

```
#chess-diagram(
  "8/5k2/8/8/3Q4/8/4K3/8",
  label-mode: "border",
  border-theme: "brown",
  size: 3.8cm,
)
```

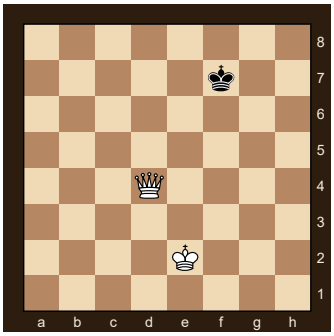


Diagram 4: Position at move 1, White to play

2 Highlights and arrows

`highlight` marks squares; each entry is a square name (drawn with `highlight-shape`, default `"filled"`), a `(square, color)` pair, or a dict `(square: .., shape: .., color: ..)` where `shape` is `"filled"`, `"cross"`, or `"circle"`.

```
#chess-diagram(
  "8/8/8/4p3/4P3/8/8/8",
  highlight: (
    "e4",
    (square: "e5", shape: "circle"),
    (square: "d5", shape: "cross"),
  ),
  size: 4cm,
)
```

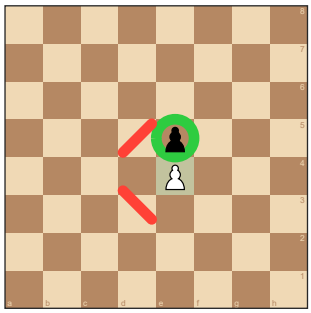


Diagram 5: Position at move 1, White to play

`arrows` draws arrows; each entry is a `(from, to)` or `(from, to, color)` tuple, or a dict `(from: .., to: .., color: ..)`. A missing color uses `arrow-color`. Arrows scale with the board and flip with it.

```
#chess-diagram(
  "rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/
RNBQKBNR",
  arrows: (
    ("e2", "e4"),
    ("g1", "f3", blue),
  ),
  size: 4cm,
)
```

Diagram 6: Position at move 1, White to play

Highlights and arrows combine freely, and a `grid: true` overlay can be added on top of any board:

```
#chess-diagram(
  "r1bqkb1r/pppp1ppp/2n2n2/4p3/2B1P3/5N2/PPPP1PPP/RNBQK2R",
  grid: true,
  highlight: ("f7",),
  arrows: (("c4", "f7"), ("f3", "e5", rgb(0, 70, 160, 200))),
  size: 4.4cm,
)
```

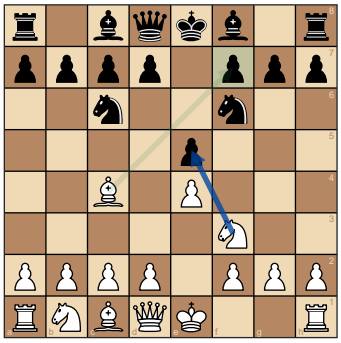


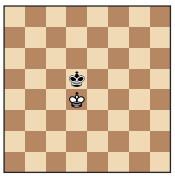
Diagram 7: Position at move 1, White to play

3 board and diagram

`board(source, ..)` draws **just the board** — no figure, no caption. It is the variant-agnostic primitive the diagram wrappers build on, and the right tool when you want a board inline in text or inside your own layout. It takes the same `source` forms and the same style overrides as `chess-diagram`.

```
A king-and-pawn ending #board(
  "8/8/8/3k4/3K4/8/8/8",
  size: 2.2cm,
  labels: false,
) drawn inline.
```

A king-and-pawn ending



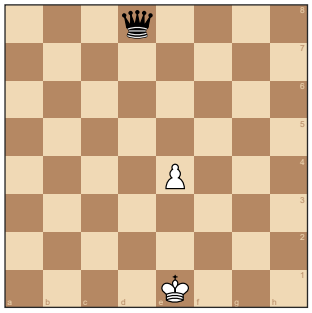
drawn inline.

Reach for `board` when you want the bare board; reach for a `*-diagram` when you want the captioned, cross-referenceable `#figure`. `chess-board` / `chess-diagram` are the **standard-variant** sugar over the generic `board` / `diagram`: same rendering, but they document the variant and reject a non-standard source.

4 position

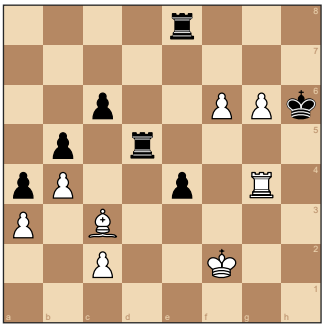
`position(..)` builds the data model — "which piece stands on which square." It accepts a FEN string (auto-detected), a **squares dict**, or a **string form**. In a squares dict the piece can be a long name, a kind abbreviation, or a bare letter (UPPER = white, lower = black):

```
#chess-diagram(
  position((
    e1: (kind: "king", color:
"white"), // long
    d8: (kind: "q", color:
"black"),    // kind
    e4:
"P",                              //
letter
  )),
  size: 4cm,
)
```



The **string form** reads like the board itself — first line is the TOP rank, `.` is empty:

```
#chess-diagram(
  position(
    "....r...",
    "........",
    "..p..PPk",
    ".p.r....",
    "pP..p.R.",
    "P.B.....",
    "..P..K..",
    "........",
  ),
  size: 4.2cm,
)
```



`position` returns a dict `(variant, cols, rows, squares, turn, castling, en-passant, halfmove, fullmove)`; `parse-fen` returns the same shape. The string form is rectangular-only and rejects characters that aren't a valid piece abbreviation or `.`.

5 Games (PGN)

`parse-pgn` returns an **array of games** (a PGN file may hold many); `.first()` takes one. Read an external file with `read` in your own file, or pass an inline raw block. The examples below assume a parsed game is in scope:

```
#let game = parse-pgn(read("game.pgn")).first()
```

Parsing is **lazy**: the roster (`tags`), the `result`, and the verbatim `movetext-raw` are extracted cheaply; the move tree is built on demand by `movetext(game)`; the engine runs only when you ask for a position. So a tournament file read only for results never tokenises movetext.

`chess-notation(game)` renders the moves the game already holds, and `board-after(game, loc)` draws the position at a locator:

```
#chess-notation(game)
```

1. e4 e5 2. Nf3 Nc6 3. Bb5 a6 4. Ba4 Nf6 5. O-O Be7 6. Re1 b5 7. Bb3 d6 8. c3 O-O

```
#board-after(game, "3w", size: 4cm)
```

Alice – Bob (2024)

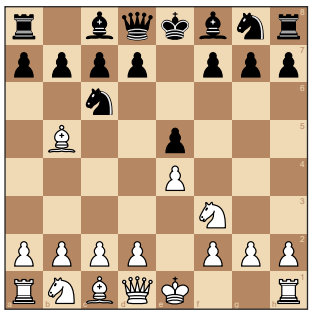


Diagram 10: Position after move 3. Bb5

5.1 Locators

A **locator** addresses one position in a game. The simple form is a string — `"30w"` / `"30b"`, the position after White's / Black's 30th **mainline** move. `position-after(game, loc)`, `board-after(game, loc, ..)`, and `to-fen(game, locator: ..)` all take it.

To address a move **inside a variation** (a PGN RAV), pass a **path** dict instead: `(line: (..hops..), at: "<final move>")`. Each hop is `(at: "<move>", into: <n>)` — branch off at mainline move `at`, descending `into` that move's variation number `n` (**0-based**: `0` is the first variation recorded at that move, `1` the second, …). The top-level `at` is where you stop within the line you reached:

```
#let g = parse-pgn(
  "[White \"V\"][Black \"T\"] 1. e4 (1.
d4 d5 2. c4) e5 *",
).first()
// into variation 0 at White's move 1
(the
// 1.d4 line), position after 2.c4:
#board-after(
  g,
  (line: ((at: "1w", into: 0),), at:
"2w"),
  size: 3.4cm,
)
```

V – T

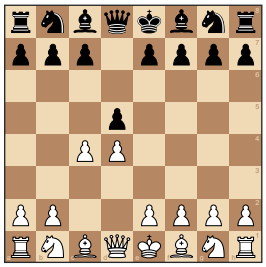


Diagram 11: Position after move 2. c4

Two easy traps:

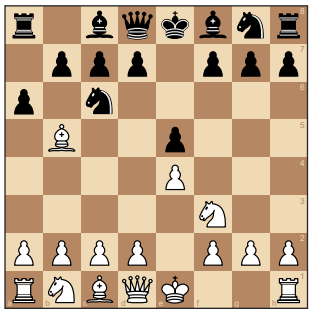
- **The trailing comma.** A single-hop path is written `((at: "1w", into: 0),)` — the comma makes it a one-element **array** of hops. Without it, Typst reads a lone parenthesised dict and the locator is malformed.
- **Mainline is the fast path.** The string form (equivalently an empty `line: ()`) indexes a memoised list of mainline positions, so many diagrams off one game stay cheap; the path form is walked move by move.

Addressing a variation that isn't there (`into:` past the recorded count) or a move past the end of its line is a hard error.

5.2 Playing moves onto a position

To explore a **new** line, or build a position from a FEN plus some moves, use `play-moves(source, moves)`. `source` is `none` (the standard start), a FEN string, or a position; `moves` is move text or a SAN array. It resolves each move against the legal moves (illegal/ambiguous is a hard error) and returns the **final** position, never mutating the source:

```
#chess-diagram(
  play-moves(none, "1. e4 e5 2. Nf3 Nc6
3. Bb5 a6"),
  size: 4cm,
)
```



5.3 Notation output

`chess-notation(source, ..)` formats move text the game already holds — no engine. `source` is a game, a move-text string, or a SAN array. It can localise the piece letters and render figurine glyphs:

```
#chess-notation(game, lang: "de")
```

1. e4 e5 2. Sf3 Sc6 3. Lb5 a6 4. La4 Sf6 5. O-O Le7 6. Te1 b5 7. Lb3 d6 8. c3 O-O

```
#chess-notation(game, figurine: true)
```

1. e4 e5 2. ♘f3 ♞c6 3. ♗b5 a6 4. ♗a4 ♞f6 5. O-O ♝e7 6. ♖e1 b5 7. ♗b3 d6 8. c3 O-O

`from` / `to` restrict output to an **inclusive** slice of moves. Both are the simple `"8b"` / `"12w"` locators; `from` defaults to the first move, `to` to the last:

```
#chess-notation(game, from: "2w", to: "3b")
```

2. Nf3 Nc6 3. Bb5 a6

Unlike `board-after`, these are **mainline-only** — they do not accept the variation **path** form, and a `from` past the end or a `to` before `from` is a hard error.

Other options: `move-numbers`, `result`, and — for a **game** source — `nags` / `comments` (consulting the PGN-handling defaults). Localization substitutes only the piece letters; files, ranks, captures, check marks and `O-O` are untouched.

5.3.1 Programmatic NAGs

`nags: true` renders the NAGs a game already carries (the `$n` glyphs in the PGN). To attach NAGs **without editing the PGN**, use `with-nags(game, map)`: it returns a **new** game whose addressed mainline moves carry the given NAGs, which `notation(.., nags: true)` then renders.

```
#let g = parse-pgn(
  "[White \"A\"][Black \"B\"] 1. e4 e5 2. Nf3 Nc6 *",
).first()
#chess-notation(
  with-nags(g, ("2w": "!", "2b": "$14")),
  nags: true,
)
```

1. e4 e5 2. Nf3! Nc6±

Each map key is a **mainline** locator (`"2w"` / `"2b"`); each value is a `"$n"` code, one of the six suffix glyphs `! ? !! ?? !? ?!` (sugar for `$1` – `$6`), or an array of those. The mapping **replaces** any NAG already

on that move, and the source game is never mutated. Only mainline moves are addressable — `notation` renders the mainline only.

5.4 Exporting FEN

`to-fen` is the inverse of `parse-fen`. It serialises a position, or a game at a locator. Standard 8×8 positions round-trip exactly:

```
#raw(to-fen(play-moves(none, "1. e4 e5 2. Nf3")))
```

```
rnbqkbnr/pppp1ppp/8/4p3/4P3/5N2/PPPP1PPP/RNBQKB1R b KQkq - 1 2
```

5.5 Drawing annotations

PGN comments can carry drawing annotations — `[%cal …]` for arrows, `[%csl …]` for highlights. Processing is **off by default**; opt in per call with `annotations: true` (or document-wide with `set-pgn-defaults`). The demo game annotates its 2nd move:

```
// move 2: {[%cal Gf3e5] [%csl Re5]}
#board-after(game, "2w", annotations:
true, size: 4cm)
```

Alice – Bob (2024)

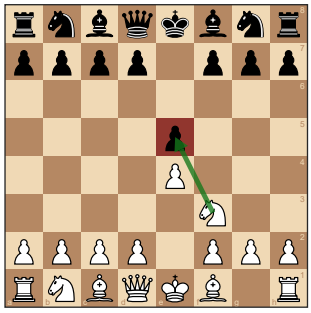


Diagram 13: Position after move 2. Nf3

The color letters (`G R Y B O`) resolve through the `annotation-colors` board style; annotations merge with any `arrows` / `highlight` you pass explicitly.

With embedded diagrams. The `diagrams` switch (PGN handling) makes `notation(diagrams: true)` splice a board after each move whose comment carries a diagram marker. If that **same** comment also holds `%cal` / `%csl` and `annotations` is on, the spliced board shows those arrows/highlights too — the two switches compose:

```
// 2. Nf3 {[%cal Gf1c4] [%csl Re5] #[After 2.Nf3]} Nc6 ...
#set-pgn-defaults(diagrams: true, annotations: true)
#chess-notation(game)   // ...text, then a board after 2.Nf3 with the arrow +
highlight
```

`diagrams` decides **whether** a board appears; `annotations` decides whether it carries the marks. The marker and the `%cal` / `%csl` must be in the **same** move's comment. (Only mainline moves are addressed — `notation` renders the mainline.)

5.6 Errors

Malformed PGN is a **hard error**: broken tag syntax and stray variation parens fail at parse time; illegal, ambiguous, or unparseable moves fail when the position is navigated. Missing Seven-Tag-Roster tags are tolerated (they default).

6 Tournament tables

Tournament tables are built from a parsed PGN's roster + results (no engine). The `*-table` renderers produce a captioned, referenceable `#figure` (kind `"chess-table"`). Each works on a list of games, with `by: "player"` or `by: "team"`. The examples use a parsed 4-player round-robin in `games`:

```
#let games = parse-pgn(read("event.pgn"))
```

`standings-table` sorts best-first (score, then tie-breaks, then first appearance):

```
#standings-table(games, by: "player")
```

Pos	Player	Pl	+	=	−	Pts	Bh	SB
1	Alice	3	2	1	0	2½	3½	2½
2	Dave	3	1	2	0	2	4	2½
3	Bob	3	1	0	2	1	5	½
4	Carol	3	0	1	2	½	5½	1

`crosstable-table` renders the round-robin grid (it **requires** a complete round-robin and errors otherwise — use standings + progress for Swiss/league):

```
#crosstable-table(games, by: "player")
```

#	Player	1	2	3	4	Pts
1	1 Alice		½	1	1	**2½**
2	2 Dave	½		1	½	**2**
3	3 Bob	0	0		1	**1**
4	4 Carol	0	½	0		**½**

`progress-table` shows the round-by-round running score (needs the `Round` tag):

```
#progress-table(games, by: "player")
```

#	Player	R1	R2	R3	Pts
1	Alice	1 (1)	1 (2)	½ (2½)	**2½**
2	Dave	½ (½)	1 (1½)	½ (2)	**2**
3	Bob	0 (0)	0 (0)	1 (1)	**1**
4	Carol	½ (½)	0 (½)	0 (½)	**½**

Each renderer takes `caption` (used by refs and the outline), `title` (a heading above the table), `supplement`, and `lang`. The compute functions (`standings`, `crosstable`, `progress`) are also public, returning plain data for custom layouts. **Team** mode groups games by the `Round = "round.board"` convention into matches.

7 Document-wide style

Styling splits into **five** buckets, each with its own setter: **board** style (`set-board-defaults`), **diagram** style (`set-diagram-defaults`), **table** style (`set-table-defaults`), **language** (`set-lang`), and **PGN handling** (`set-pgn-defaults`). `set-chess-defaults` is the single **umbrella** over all five: it routes each key to the bucket that owns it, so any settable default can go through it (`set-chess-defaults(dark: blue, lang: "de", nags: true)` is fine). A setter affects **every subsequent** diagram/table; per-call arguments still override.

```
#set-board-defaults(light: rgb("#eeeed2"), dark: rgb("#769656"), size: 5cm)
#set-diagram-defaults(info-bold: false, supplement: [Position])
#set-table-defaults(supplement: [Tabelle])
```

```
#set-piece-set("merida")                          // sugar for set-board-defaults
#set-chess-defaults(dark: blue, info-gap: 1em)    // umbrella
```

Every default is equivalently a **per-call** argument — the same green theme, set once above vs. passed to one diagram:

```
#chess-diagram(
  "rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/
RNBQKBNR",
  light: rgb("#eeeed2"), dark:
rgb("#769656"),
  size: 4cm,
)
```

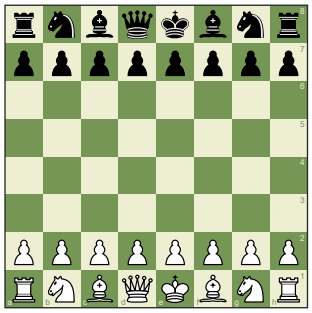


Diagram 14: Position at move 1, White to play

`flip` is the one setting **not** allowed in any defaults setter — orientation is a per-diagram choice, so `set-chess-defaults(flip: ..)` is an error. (`supplement` and `outline-title` live in **both** the diagram and table buckets; the umbrella routes them to **diagram** — use `set-table-defaults` for the table ones.)

7.1 Language

A single document **language** drives every language-aware string — diagram and table supplements, outline titles, and notation piece letters. Default is English; `"auto"` follows `#set text(lang: ..)`; or pick a code:

```
#set-lang("de")      // Diagramm / Tabelle / Sf3, Lb5, ...
#set-lang("auto")    // follow #set text(lang: ..)
```

Every localizable string is also per-call overridable (the `lang:` argument seen on `chess-notation` above). Adding a language is a no-code change: drop a `src/assets/i18n/<code>.typ` and register it in `src/i18n.typ`.

7.2 PGN handling

A **PGN-handling** bucket decides how much of a parsed game's embedded extras get interpreted at render time. Parsing stays lossless; these switches only decide what is processed, and **all default off**:

```
#set-pgn-defaults(annotations: true, nags: true, comments: true)
```

key	effect
`annotations`	`%cal` / `%csl` → arrows/highlights on `board-after`
`nags`	render NAGs (`Nf3!`, `d4±`) in `notation`
`comments`	include comment prose in `notation`
`diagrams`	embed a board in `notation` after each move marked for one

Each is also a per-call argument (`auto` → the document default) on `notation` / `board-after` — as used in the annotations example above.

8 Outlines and references

Diagrams and tables each carry a distinct figure `kind` (`"chess"` / `"chess-table"`), so they get their own counters, can be **referenced** (`@label` → "Diagram 3" / "Table 2"), and can be **listed separately**:

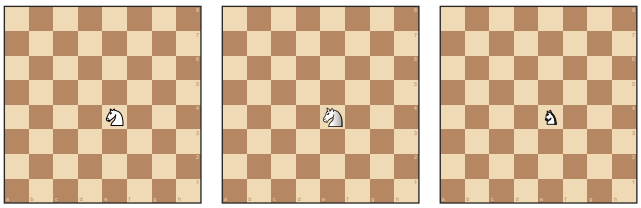
```
#chess-diagram-outline()      // list of chess diagrams
#chess-table-outline()        // list of tournament tables
#chess-outlines()             // both, diagrams then tables

#chess-diagram(starting-fen, caption: [Start]) <start>
As shown in @start, ...
```

Only figures that carry a **caption** appear in an outline (a caption-less figure is still referenceable but unlisted, matching Typst). Titles are language-aware by default and settable per call (`title:` , `lang:`) or document-wide (`set-diagram-defaults(outline-title: ..)`).

9 Pieces and fonts

Pieces are drawn from bundled **SVG piece sets**. Two ship with the package, both GPLv2+: `cburnett` (default) and `merida` . Choose one per diagram with `piece-set:` or document-wide with `set-piece-set` :

```
#grid(columns: 3, gutter: 8pt, align: bottom,
  board("8/8/8/8/4N3/8/8/8", size: 2.6cm, piece-set: "cburnett"),
  board("8/8/8/8/4N3/8/8/8", size: 2.6cm, piece-set: "merida"),
  board("8/8/8/8/4N3/8/8/8", size: 2.6cm, piece-set: "unicode"),
)
```

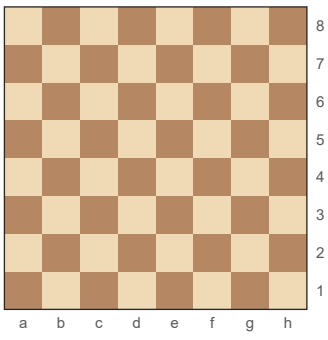


The renderer accepts **any** set name and loads `src/assets/piece_sets/<name>/{w,b}{K,Q,R,B,N,P}.svg` on demand, so you can add a set by dropping such a folder into your copy and passing its name. `piece-set:` (or `none`) selects the glyph fallback `"unicode"` — solid Unicode chess glyphs distinguished by fill and a contrasting stroke; it needs a font carrying them.

10 Coordinates

Files `a` – `h` , ranks `1` – `8` ; `a1` is the dark square in the lower-left corner, `h8` the upper-right. Square names are case-insensitive (`"E4"` = `"e4"`).

```
#board(
  "8/8/8/8/8/8/8/8",
  label-mode: "outside",
  size: 4cm,
)
```



11 Sizing

The default board size suits an A4 two-column layout. A diagram reads the available space at its insertion point via `layout` , so it adapts to any column or page size without being told the geometry, and shrinks to fit if asked for more than fits. `size` may be a `length` , a `ratio` of the available width, or `auto` :

```
#chess-diagram(
  "8/8/8/3k4/3K4/8/8/8",
  size: 60%,   // of the available
width
)
```

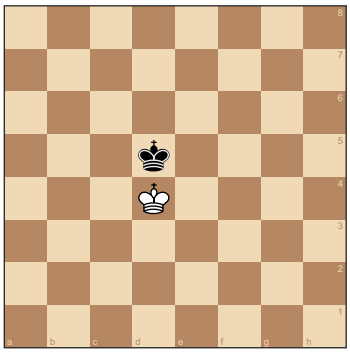


Diagram 15: Position at move 1, White to play